```
int ci = sequence++ % PIPE_DEPTH;
sem_wait(&camera_free[ci]);              // 等 JPEG 线程用完这个 fd 上一次的槽
int n = read(fd[ci], &job.hdr, sizeof(job.hdr));   // 只取帧头；内核归还旧槽、给出新槽
job.camera_index = ci;
raw_push(&job);

// JPEG 线程
struct raw_job raw = raw_pop();
const uchar *pixels = camera_dma + raw.hdr.reserved * CAM_SLOT_BYTES;
camproc_jpeg_raw(encoded_buf[bi], JPEG_CAP, &raw.hdr, pixels, half, wb, quality, &out.len);
sem_post(&camera_free[raw.camera_index]);  // 槽用完了，这个 fd 可以再 read
```

JPEG 结果只有几十 KB，主线程把最新一帧复制一份留给 `/snapshot.jpg`；大块的 RAW 数据始终留在 DMA 槽里。

方法三：有界队列与背压。 流水线的各段速度不同，必须有机制防止快的一段把慢的一段淹没。这里全部用计数信号量实现（表 7.23）。

表 7.23: 流水线中的同步对象

对象	初值	含义
`pipeline_start`	0	主线程有了第一个观看者时 `sem_post`，唤醒采集线程打开摄像头
`camera_free[0..1]`	1, 1	每个 camera fd 持有的 DMA 槽是否已被 JPEG 线程用完
`raw_free` / `raw_ready`	2 / 0	RAW 队列的空位 / 待编码的帧
`encoded_free` / `encoded_ready`	2 / 0	JPEG 缓冲的空位 / 待发送的帧
`convert_lock`	互斥锁	`camproc.c` 复用静态的 RAW 解包缓冲，单次快照不能与 JPEG 线程同时转换

两个队列都是单生产者、单消费者的环形缓冲。生产者写好元素后用 release 语义推进尾指针，消费者用 acquire 语义读，因此不需要额外的锁：

```
static void raw_push(const struct raw_job *job) {
  sem_wait(&raw_free);                                   // 背压：队列满就等
  uint tail = __atomic_load_n(&raw_tail, __ATOMIC_RELAXED);
  rawq[tail % PIPE_DEPTH] = *job;
  __atomic_store_n(&raw_tail, tail + 1, __ATOMIC_RELEASE);   // 先写元素，再发布
  sem_post(&raw_ready);
}
```

如果发送慢了（例如某个观看者网速很差，`write()` 阻塞），`encoded_free` 很快耗尽，JPEG 线程停在 `sem_wait()` 上；它不再归还 `camera_free`，采集线程也随之停下；内核那边两个槽都被持有，Unicam 自动把多余的帧写进 dummy 丢弃。慢的一端就这样一级一级地把压力传回传感器，而不会让内存无限增长。

观看者全部离开时，主线程把 `pipeline_run` 清零，采集线程退出内层循环，等两个 `camera_free` 都收回之后才关闭两个 fd（这时 JPEG 线程已不再读映射），最后一个引用关闭使内核停流断电。每次重新开始推流时 `pipeline_generation` 加 1，主线程丢弃上一代残留在队列里的帧。

方法四：减少每帧的工作量。 流水线只能把周期压到最慢的一段；要再快，就得让最慢的那段少干活。现有的两个开关作用在不同的阶段：

- `-s`（320×240）：每个 2×2 Bayer 块直接合成一个像素，省掉双线性去马赛克，输出像素减少 75%，同时减轻 JPEG 和发送两段；
- `-q 60`：降低 JPEG 质量，量化更粗，码流更短，主要减轻发送一段，对编码时间影响较小。

网络推流建议从 `camserver -s -q 60` 开始。